

SKILL_WEEK - 06 (SESSION NUMBER - 11)

Task 1: To store commands, maintain history buffer, handle capacity limits, retrieve entries, display history, and validate consistency.

```
UW PICO 5.09
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_HISTORY 5
#define MAX_COMMAND 100

int main()
{
    char history[MAX_HISTORY][MAX_COMMAND];
    char command[MAX_COMMAND];

    int count = 0;

    while (1)
    {
        printf("\nmyshell> ");

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            printf("Exiting history program...\n");
            break;
        }

        if (strcmp(command, "history") == 0)
        {
            printf("\nCommand History:\n");

            if (count == 0)
            {
                printf("History is empty.\n");
            }
            else
            {
                for (int i = 0; i < count; i++)
                {
                    printf("%d. %s\n", i + 1, history[i]);
                }
            }

            continue;
        }

        if (strncmp(command, "get ", 4) == 0)
        {
            int index = atoi(command + 4);

            if (index >= 1 && index <= count)
            {
                printf("Retrieved Command: %s\n",

```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/session11_task1.c -o bin/session11_task1
amrutha@Amruthas-MacBook-Air OSSP % ./bin/session11_task1

myshell> ls
Command stored successfully.

myshell> pwd
Command stored successfully.

myshell> date
Command stored successfully.

myshell> history

Command History:
1. ls
2. pwd
3. date

myshell> get 2
Retrieved Command: pwd

myshell> exit
Exiting history program...
amrutha@Amruthas-MacBook-Air OSSP %
```

Task 2: To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

```
UW PICO 5.09

#include <stdio.h>
#include <string.h>

#define MAX_COMMANDS 5
#define MAX_LENGTH 100

int main()
{
    char pipeline[MAX_COMMANDS][MAX_LENGTH];
    int count;

    printf("Enter number of commands in pipeline: ");
    scanf("%d", &count);
    getchar();

    if (count <= 0 || count > MAX_COMMANDS)
    {
        printf("Invalid pipeline size.\n");
        return 1;
    }

    for (int i = 0; i < count; i++)
    {
        printf("Enter command %d: ", i + 1);
        fgets(pipeline[i], sizeof(pipeline[i]), stdin);
        pipeline[i][strcspn(pipeline[i], "\n")] = '\0';
    }

    printf("\nPipeline Structure:\n");

    for (int i = 0; i < count; i++)
    {
        printf("%s", pipeline[i]);

        if (i != count - 1)
            printf(" | ");
    }

    printf("\n\nExecution Order:\n");

    for (int i = 0; i < count; i++)
    {
        printf("Process %d executes: %s\n",
            i + 1, pipeline[i]);
    }

    printf("\nPipeline layout validated successfully.\n");

    return 0;
}
```

OUTPUT:

```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/session11_task2
Enter number of commands in pipeline: 3
Enter command 1: ls -l
Enter command 2: grep .c
Enter command 3: wc -l

Pipeline Structure:
ls -l | grep .c | wc -l

Execution Order:
Process 1 executes: ls -l
Process 2 executes: grep .c
Process 3 executes: wc -l

Pipeline layout validated successfully.
```

(SESSION NUMBER - 12)

Task 1: To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

```
UW PICO 5.09

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t p1, p2;

    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    p1 = fork();
    if (p1 == 0)
    {
        dup2(fd[1], STDOUT_FILENO);
        close(fd[0]);
        close(fd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp");
        exit(1);
    }

    p2 = fork();
    if (p2 == 0)
    {
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        close(fd[1]);

        execlp("grep", "grep", ".c", NULL);

        perror("execlp");
        exit(1);
    }

    close(fd[0]);
    close(fd[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);

    printf("\nPipeline execution completed successfully.\n");

    return 0;
}
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/session12_task1.c -o bin/session12_task1
amrutha@Amruthas-MacBook-Air OSSP % ./bin/session12_task1
drwxr-xr-x  4 amrutha  staff   128 Jul 31 14:05 docs
-rw-r--r--  1 amrutha  staff  1507 Aug 14 19:15 double_quote_flowchart.txt
-rw-r--r--  1 amrutha  staff  1967 Aug 15 10:40 escape_flowchart.txt
drwxr-xr-x  6 amrutha  staff   192 Jul 31 14:05 include
-rw-r--r--  1 amrutha  staff   174 Sep 23 10:20 lowlevel_copy.txt
-rw-r--r--  1 amrutha  staff  1177 Aug 14 17:11 parser_flowchart.txt
-rw-r--r--  1 amrutha  staff  1207 Aug 15 15:01 process_flowchart.txt
-rw-r--r--  1 amrutha  staff  1277 Aug 14 17:42 quote_flowchart.txt
drwxr-xr-x  3 amrutha  staff    96 Jul 31 14:05 scripts
-rw-r--r--  1 amrutha  staff  1441 Sep 28 09:35 skill10_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff  1862 Sep 28 09:45 skill10_task2_flowchart.txt
-rw-r--r--  1 amrutha  staff  1880 Sep 21 11:00 skill5_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff  2013 Sep 21 11:12 skill5_task2_flowchart.txt
-rw-r--r--  1 amrutha  staff  1902 Sep 21 11:32 skill6_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff  1611 Sep 21 11:50 skill6_task2_flowchart.txt
-rw-r--r--  1 amrutha  staff  2164 Sep 21 12:11 skill7_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff  2036 Sep 21 12:28 skill7_task2_flowchart.txt
-rw-r--r--  1 amrutha  staff  1829 Sep 21 14:41 skill8_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff  2208 Sep 21 14:55 skill8_task2_flowchart.txt
-rw-r--r--  1 amrutha  staff  2613 Sep 28 09:03 skill9_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff  1821 Sep 28 09:17 skill9_task2_flowchart.txt
-rw-r--r--  1 amrutha  staff   174 Sep 23 10:19 source.txt
drwxr-xr-x 46 amrutha  staff  1472 Sep 28 15:45 src
-rw-r--r--  1 amrutha  staff   174 Sep 23 10:20 stdio_copy.txt
-rw-r--r--  1 amrutha  staff  1702 Aug 17 10:43 wait_flowchart.txt
-rw-r--r--  1 amrutha  staff  1893 Sep 23 11:09 week10_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff  1793 Sep 23 11:30 week10_task2_flowchart.txt
-rw-r--r--  1 amrutha  staff  1848 Aug 24 20:52 week3_task3_flowchart.txt
-rw-r--r--  1 amrutha  staff  1246 Aug 24 21:20 week3_task4_flowchart.txt
-rw-r--r--  1 amrutha  staff  2036 Sep 19 14:06 week5_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff  1760 Sep 19 14:17 week5_task2_flowchart.txt
-rw-r--r--  1 amrutha  staff  1760 Sep 19 14:52 week6_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff  1520 Sep 19 15:14 week6_task2_flowchart.txt
-rw-r--r--  1 amrutha  staff  1594 Sep 19 17:26 week7_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff  1494 Sep 19 17:53 week7_task2_flowchart.txt
-rw-r--r--  1 amrutha  staff  1328 Sep 20 21:38 week8_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff  1726 Sep 20 21:48 week8_task2_flowchart.txt
-rw-r--r--  1 amrutha  staff  2027 Sep 23 10:24 week9_task1_flowchart.txt
-rw-r--r--  1 amrutha  staff  1494 Sep 23 10:36 week9_task2_flowchart.txt
```

Task 2: To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{
    int pipe1[2], pipe2[2];
    pid_t p1, p2, p3;

    pipe(pipe1);
    pipe(pipe2);

    p1 = fork();

    if (p1 == 0)
    {
        dup2(pipe1[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("ls", "ls", "-l", NULL);
        exit(1);
    }

    p2 = fork();

    if (p2 == 0)
    {
        dup2(pipe1[0], STDIN_FILENO);
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("grep", "grep", ".c", NULL);
        exit(1);
    }

    p3 = fork();

    if (p3 == 0)
    {
        dup2(pipe2[0], STDIN_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);
    }
}
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/session12_task2.c -o bin/session12_task2
amrutha@Amruthas-MacBook-Air OSSP % ./bin/session12_task2
39
Long pipeline executed successfully.
amrutha@Amruthas-MacBook-Air OSSP %
```